Московский государственный университет имени М.В. Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра суперкомпьютеров и квантовой информатики

Майстренко Марат Алексеевич

Параллельное выполнение алгоритма решения СЛАУ методом отражений

ОТЧЁТ ПО ВЫПОЛНЕНИЮ ЗАДАНИЯ №2

Москва, 2026

Оглавление

Алгоритм решения СЛАУ методом отражения 3

Алгоритм метода отражений 4

Постановка задачи 6

Реализация поставленной задачи 7

Тестовый интерфейс 10

Результаты 10

Сравнение OMP и MPI версии программы 14

Заключение 16

Алгоритм решения СЛАУ методом отражения

Определение. Пусть дан единичный вектор **x** размерности n, (т.е. $||\mathbf{x}|| = 1$), тогда матрицей отражения называется $U(\mathbf{x}) = \mathbf{I} - 2\mathbf{x}\mathbf{x}^*$,

где * - транспонирование, норма - евклидова норма.

Лемма 1. Матрица $U(\mathbf{x})$ является ортогональной матрицей.

Лемма 2. Матрица $U(\mathbf{x})$ является симметричной матрицей.

Лемма 3. Матрица $U(\mathbf{x})$ имеет собственное значение -1 кратности 1, которому отвечает собственный вектор **x** и собственное значение 1 кратности n-1, которому отвечает собственное подпространство $\langle\mathbf{x}\rangle^{\perp} = \{\mathbf{y}: (\mathbf{x},\mathbf{y}) = 0\}$.

Лемма 4. Геометрический смысл преобразования, задаваемый матрицей $U(\mathbf{x})$: отражение относительно гиперплоскости $\langle\mathbf{x}\rangle^{\perp}$

Лемма 5. Пусть **e** - единичный вектор ($||\mathbf{e}||=1$), тогда для любого n-мерного вектора **y** существует такой единичный вектор **x**, что $U(\mathbf{x})\mathbf{y} = ||\mathbf{y}||\mathbf{e}$.

Лемма 6. Произведение матрицы отражения на вектор может быть вычислено за 2n-1 операцию сложения и 2n+1 операцию умножения.

Лемма 7. Произведение матрицы отражения (размера n x n) на матрицу размера n x m может быть вычислено за m(2n-1) операций умножения и m(2n+1) операций сложения.

Алгоритм метода отражений

Пусть требуется решить систему линейных уравнений **Ax**=**b**

Через $\mathbf{a}_1 = (a_{11},...,a_{1n})^*$ обозначим первый столбец матрицы **A**, согласно лемме 5 существует вектор $\mathbf{x}^{(1)}$, равный,

$$x^{(1)} = \pm \frac{a_1 - \|a_1\| e_1}{\|a_1 - \|a_1\| e_1\|}$$

такой что $U(\mathbf{x}^{(1)})\mathbf{a}_1 = \|\mathbf{a}_1\|\mathbf{e}_1$. Умножим на матрицу отражения левую и правую части уравнения **Ax**=**b**, получим уравнение $\mathbf{A}^{(1)}\mathbf{x}=\mathbf{b}^{(1)}$

$$A^{(1)} = U_1 A = U\left(x^{(1)}\right) A, \quad A^{(1)} = \begin{pmatrix} \|a_1\| & c_{12} & \dots & c_{1n} \\ 0 & a_{22}^{(1)} & \dots & a_{1n}^{(1)} \\ \dots & \dots & \ddots & \dots \\ 0 & a_{n2}^{(1)} & \dots & a_{nn}^{(1)} \end{pmatrix}, \quad b^{(1)} = U_1 b$$

Пусть сделан k-1 шаг алгоритма, и система приведена к виду $\mathbf{A}^{(k-1)}\mathbf{x} = \mathbf{b}^{(k-1)}$, где $\mathbf{A}^{(k-1)} = U_{k-1}U_{k-2}...U_1\mathbf{A}$, а $\mathbf{b}^{(k-1)} = U_{k-1}U_{k-2}...U_1\mathbf{b}$.

$$A^{(k-1)} = \begin{pmatrix} \|a_1\| & c_{12} & c_{13} & \dots & c_{1,k-1} & c_{1k} & \dots & c_{1n} \\ 0 & \|a_2\| & c_{23} & \dots & c_{2,k-1} & c_{2k} & \dots & c_{2n} \\ 0 & 0 & \|a_3\| & \dots & c_{3,k-1} & c_{3k} & \dots & c_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \|a_{k-1}\| & c_{k-1,k} & \dots & c_{k-1,n} \\ 0 & 0 & 0 & \dots & 0 & a_{kk}^{(k-1)} & \dots & a_{kn}^{(k-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 0 & a_{nk}^{(k-1)} & \dots & a_{nn}^{(k-1)} \end{pmatrix}, \quad U_i = \begin{pmatrix} I_{i-1} & 0 \\ 0 & U\left(x^{(i)}\right) \end{pmatrix}, \quad i=1,\dots,k-1$$

Обозначим через $\mathbf{a}_k = (a_{kk}, \dots, a_{nk})^*$. Тогда согласно лемме 5 существует вектор $\mathbf{x}^{(k)}$ (размерности n-k+1) равный

$$x^{(k)} = \pm \frac{a_k - \|a_k\| e_k}{\|a_k - \|a_k\| e_k\|}$$

такой, что $U(\mathbf{x}^{(k)})\mathbf{a}_k = \|\mathbf{a}_k\|\mathbf{e}_k$, где $\mathbf{e}_k = (1,0,\dots,0)$ (вектор размерности n-k+1)

Умножая матрицу U_k на левую и правую часть системы уравнений $\mathbf{A}^{(k-1)}\mathbf{x} = \mathbf{b}^{(k-1)}$, получим систему уравнений $\mathbf{A}^{(k)}\mathbf{x} = \mathbf{b}^{(k)}$, где $\mathbf{A}^{(k)} = U_k U_{k-1} U_{k-2} \cdots U_1 \mathbf{A}$, а $\mathbf{b}^{(k)} = U_k U_{k-1} U_{k-2} \cdots U_1 \mathbf{b}$.

$$A^{(k-1)} = \begin{pmatrix} \|a_1\| & c_{12} & c_{13} & \dots & c_{1,k-1} & c_{1k} & c_{1,k+1} & \dots & c_{1n} \\ 0 & \|a_2\| & c_{23} & \dots & c_{2,k-1} & c_{2k} & c_{2,k+1} & \dots & c_{2n} \\ 0 & 0 & \|a_3\| & \dots & c_{3,k-1} & c_{3k} & c_{3,k+1} & \dots & c_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \|a_{k-1}\| & c_{k-1,k} & c_{k-1,k+1} & \dots & c_{k-1,n} \\ 0 & 0 & 0 & \dots & 0 & \|a_k\| & c_{k,k+1} & \dots & c_{k,n} \\ 0 & 0 & 0 & \dots & 0 & 0 & a^{(k)}_{k+1,k+1} & \dots & a^{(k)}_{k+1,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 0 & 0 & a^{(k)}_{n,k+1} & \dots & a^{(k)}_{nn} \end{pmatrix}, \quad U_k = \begin{pmatrix} I_{k-1} & 0 \\ 0 & U(x^{(k)}) \end{pmatrix}$$

Отметим, что при умножении матрицы вида U_k на матрицу вида $\mathbf{A}^{(k-1)}$ происходит умножение матрицы отражения на подматрицу матрицы $\mathbf{A}^{(k-1)}$ размера n-k+1 (остальная часть матрицы в преобразовании не участвует).

После n шагов процесса система примет вид $\mathbf{Rx} = \mathbf{y}$, где $\mathbf{R}$ - верхнетреугольная матрица. $R = \mathbf{A}^{(n)} = U_n U_{n-1} \cdots U_2 U_1 \mathbf{A}$, $\mathbf{y} = \mathbf{b}^{(n)} = U_n U_{n-1} \cdots U_2 U_1 \mathbf{b}$. Полученную систему можно решить используя обратный ход метода Гаусса.

Постановка задачи

Требуется написать параллельную программу с использованием технологии MPI для решения системы линейных уравнений **Ax**=**b** методом отражений.

Использование кластера **Polus** в этом задании **обязательно**.

Требования к программе:

1) Матрица **A** считается достаточно большой, но можно считать, что памяти одного узла достаточно для хранения всей матрицы.

2) Матрица **A** и вектор правой части **b** генерируется при помощи формулы, выдаваемой преподавателем.

3) Программа должна выводить норму невязки ||**Ax**-**b**||

4) В случае известного точного решения системы линейных уравнений программа должна выводить норму разницы между точным и полученным решением.

5) Программа должна выводить время, затраченное на приведение системы к верхнетреугольному виду (T_1), время, затраченное программой на обратный ход методом Гаусса (T_2), и общее время ($T_{all} = T_1+T_2$)

6) Программа должна корректно вычислять решение системы уравнений, при этом загрузка нитей должна быть максимально равномерной.

Реализация поставленной задачи

При разработке параллельной MPI-программы для решения СЛАУ методом отражений Хаусхолдера реализован ряд алгоритмических оптимизаций, обеспечивающих эффективное масштабирование на распределенной памяти.

Циклическое распределение столбцов блоками

Матрица распределяется по процессам не построчно, а постолбцово блоками размера block. Циклическая схема owner(j)= (j/block)%p обеспечивает перемешивание столбцов между процессами. Это критически важно для прямого хода метода Хаусхолдера: на поздних шагах k количество активных столбцов уменьшается, и при непрерывном распределении большая часть нагрузки легла бы на один процесс. Циклическое распределение гарантирует, что на каждом шаге активные столбцы остаются равномерно распределенными между всеми процессами, предотвращая дисбаланс загрузки и простой вычислительных ресурсов.

Упакованный буфер для вектора Хаусхолдера

В классической реализации метода отражений вектор u, его норма и обратная норма передаются отдельными MPI-сообщениями. В данной программе реализован прием упаковки: все необходимые данные размещаются в едином буфере [norm, inv_norm, u[0..tail-1]], который рассылается одним вызовом MPI_Bcast. Это сокращает количество коммуникационных операций на каждом шаге с трех до одной, существенно снижая накладные расходы на латентность сети при большом количестве итераций (n до 8000).

Распределенный обратный ход без сбора матрицы.

Традиционный подход к обратной подстановке в MPI-программах требует сбора всей треугольной матрицы на корневом процессе, что создает узкое место как по памяти ($O(n^2)$ на одном узле), так и по времени. В реализованном алгоритме обратный ход выполняется распределенно: матрица остается на тех процессах, где она была построена. На каждом шаге решения системы процессы вычисляют локальный вклад в правую часть от уже найденных компонент решения x_j, затем с помощью MPI_Reduce суммируют его на процессе-владельце текущего блока строк. Владелец решает свою часть системы и рассылает результат через MPI_Bcast. Такой подход исключает необходимость хранения полной матрицы на одном процессе и позволяет параллельно выполнять вычисления на всех узлах вплоть до получения окончательного решения.

Согласованность распределения матрицы и обратного хода.

Ключевым моментом является то, что циклическая схема распределения столбцов в прямом ходе owner(j) = (j / block) % p порождает аналогичное циклическое распределение строк диагональных блоков в обратном ходе block_owner = blk % p. Это не случайное совпадение, а намеренное проектирование: процесс, владеющий столбцом k в прямом ходе, владеет строкой k в обратном, что минимизирует пересылки данных между этапами алгоритма и сохраняет локальность данных.

Адаптивный выбор размера блока.

Размер блока block не фиксирован, а вычисляется функцией choose_block_size на основе формулы: целевое количество блоков устанавливается как 8 * p, что обеспечивает достаточную для балансировки нагрузки, но ограничивает размер блока в диапазоне [16, 64] для эффективного использования кэш-памяти.

Параметры запуска

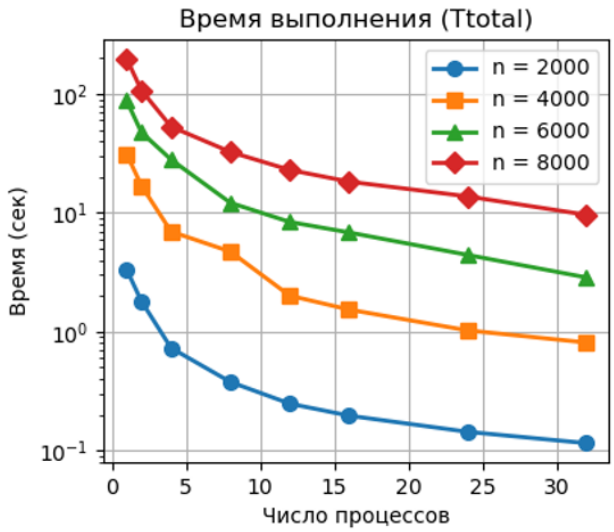
```
#BSUB -n 32
#BSUB -W 00:30
#BSUB -o results_32.txt
#BSUB -m "polus-c3-ib polus-c4-ib"
#BSUB -R "span[ptile=16]"
#BSUB -R "affinity[core(1)]"
mpiexec ./zad_2
```

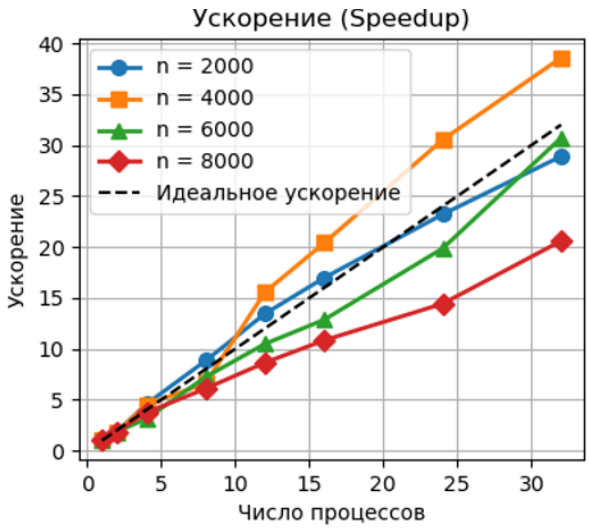
Запуск программы производится через систему LSF. В командном файле с помощью директивы #BSUB -n 32 запрашивается 32 вычислительных ядра. Параметр #BSUB -m "polus-c3-ib polus-c4-ib" ограничивает запуск задачи двумя указанными вычислительными узлами. Директива #BSUB -R "span[ptile=16]" распределяет запрошенные ядра по узлам равномерно: по 16 ядер на каждом узле. Параметр #BSUB -R "affinity[core(1)]" включает привязку MPI-процессов к вычислительным ядрам, закрепляя каждый процесс за отдельным физическим ядром и предотвращая его миграцию между ядрами во время выполнения. При запуске создаётся 32 MPI-процесса, и каждый процесс закрепляется за своим ядром - по 16 процессов на каждом из двух узлов.

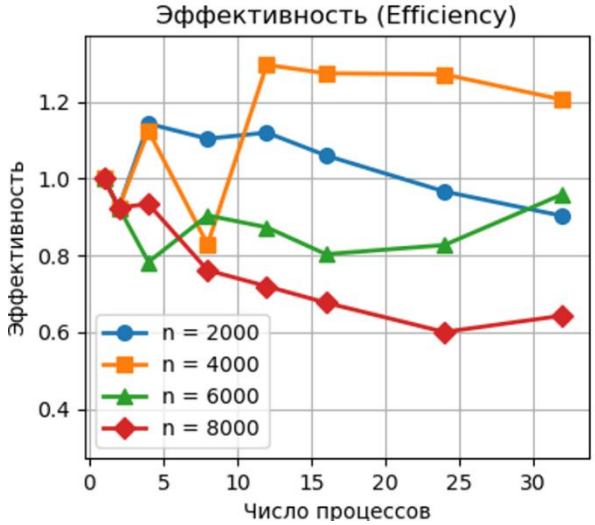
Тестовый интерфейс

Вывод: время приведения к треугольному виду T_1, время обратного хода T_2, общее время решения T_{all}, норма невязки ||Ax-b|| и норма разницы с точным решением ||x - x_true||.

Результаты







Для всех четырёх размеров матрицы наблюдается устойчивое уменьшение времени решения при увеличении числа MPI-процессов вплоть до 32 процессов, причём насыщение производительности не достигается в рассмотренном диапазоне.

Наибольший прирост производительности наблюдается при увеличении числа процессов от 1 до 8-16: для n=2000 время решения уменьшается с 3.34 до 0.38 секунд (ускорение около 8.8 раз), для n=4000 - с 31.31 до 4.72 секунд (ускорение около 6.6 раз), для n=6000 - с 87.93 до 12.15 секунд (ускорение около 7.2 раз), для n=8000 - с 198.37 до 32.56 секунд (ускорение около 6.1 раз).

При дальнейшем увеличении числа процессов до 32 ускорение продолжает расти: для n=2000 время снижается до 0.12 секунд (общее ускорение около 28.9 раз относительно одного процесса), для n=4000 - до 0.81 секунд (ускорение около 38.6 раз), для n=6000 - до 2.86 секунд (ускорение около 30.7 раз), для n=8000 - до 9.63 секунд (ускорение около 20.6 раз относительно одного процесса и около 11.1 раз относительно двух процессов).

Сравнение с предыдущей реализацией демонстрирует кардинальное улучшение производительности. Для n=2000 на 32 процессах время решения сократилось с 1.69 до 0.12 секунд (ускорение в 14.7 раз), для n=4000 - с 12.35 до 0.81 секунд (ускорение в 15.2 раз), для n=6000 - с 41.41 до 2.86 секунд (ускорение в 14.5 раз). Достигнутое улучшение является прямым следствием устранения кэш-промахов в прямом ходе метода Хаусхолдера за счёт оптимизации порядка обхода столбцов и использования упакованного буфера для передачи вектора отражения.

При этом время обратного хода (T2) остаётся на два-три порядка меньше времени прямого хода (T1) и растёт незначительно: от 0.007 секунд на одном процессе до 0.001-0.011 секунд на 32 процессах в зависимости от размерности матрицы. Распределённая обратная подстановка без сбора полной матрицы на корневом процессе обеспечивает масштабируемость и этого этапа, хотя его вклад в общее время остаётся пренебрежимо малым.

Невязка решения и ошибка относительно точного решения остаются стабильными для всех конфигураций: невязка составляет порядка 10^{-9}–10^{-7}, ошибка - 10^{-12}–10^{-11}, что подтверждает численную устойчивость реализованного алгоритма и отсутствие влияния параллельной декомпозиции на точность вычислений.

Threads	**T_1 (сек.)**	**T_2 (сек.)**	**T_{all} (сек.)**	**Невязка**	**Ошибка**
1	3.329804	0.007233	3.337037	3.710000e-09	1.855000e-12
2	1.799894	0.003910	1.803804	3.717382e-09	1.857804e-12
4	0.727618	0.001981	0.729599	3.719490e-09	1.858885e-12
8	0.376024	0.001746	0.377769	3.713022e-09	1.855539e-12
12	0.246796	0.001465	0.248261	3.708229e-09	1.853164e-12
16	0.195026	0.001623	0.196649	3.707361e-09	1.852628e-12
24	0.142168	0.001643	0.143811	3.708438e-09	1.853127e-12
32	0.113820	0.001648	0.115468	3.707537e-09	1.852655e-12

Таблица №1. Результаты эксперимента с n = 2000

Threads	**T_1 (сек.)**	**T_2 (сек.)**	**T_{all} (сек.)**	**Невязка**	**Ошибка**
1	31.279641	0.029515	31.309156	2.150000e-08	5.350000e-12
2	16.907914	0.015954	16.923868	2.172407e-08	5.435461e-12
4	6.948817	0.008877	6.957694	2.133840e-08	5.338809e-12
8	4.717222	0.005238	4.722460	2.135166e-08	5.341240e-12
12	2.008903	0.003316	2.012218	2.132213e-08	5.334152e-12
16	1.531492	0.003851	1.535343	2.131220e-08	5.331676e-12
24	1.021359	0.004431	1.025790	2.131684e-08	5.332744e-12
32	0.807779	0.003661	0.811440	2.131451e-08	5.332110e-12

Таблица №2. Результаты эксперимента с n = 4000

Threads	T_1 (сек.)	T_2 (сек.)	T_{all} (сек.)	Невязка	Ошибка
1	87.881423	0.052866	87.934289	7.150000e-08	1.190000e-11
2	47.503472	0.028576	47.532048	7.234710e-08	1.205758e-11
4	28.051289	0.019836	28.071125	7.138269e-08	1.189724e-11
8	12.142317	0.009785	12.152101	7.128772e-08	1.188155e-11
12	8.381616	0.006996	8.388612	7.128551e-08	1.188102e-11
16	6.833334	0.006631	6.839965	7.128666e-08	1.188120e-11
24	4.423025	0.006177	4.429203	7.129296e-08	1.188220e-11
32	2.855704	0.007474	2.863178	7.129019e-08	1.188192e-11

Таблица №3. Результаты эксперимента с n = 6000

Threads	T_1 (сек.)	T_2 (сек.)	T_{all} (сек.)	Невязка	Ошибка
1	198.281490	0.091971	198.373461	1.200000e-07	1.500000e-11
2	107.179184	0.049714	107.228898	1.220913e-07	1.525938e-11
4	53.055622	0.037465	53.093087	1.199011e-07	1.498530e-11
8	32.539674	0.017845	32.557519	1.198693e-07	1.498157e-11
12	22.944949	0.018573	22.963522	1.197779e-07	1.496962e-11
16	18.298745	0.010629	18.309374	1.197565e-07	1.496706e-11
24	13.746873	0.008425	13.755299	1.197932e-07	1.497172e-11
32	9.622943	0.010945	9.633888	1.197884e-07	1.497121e-11

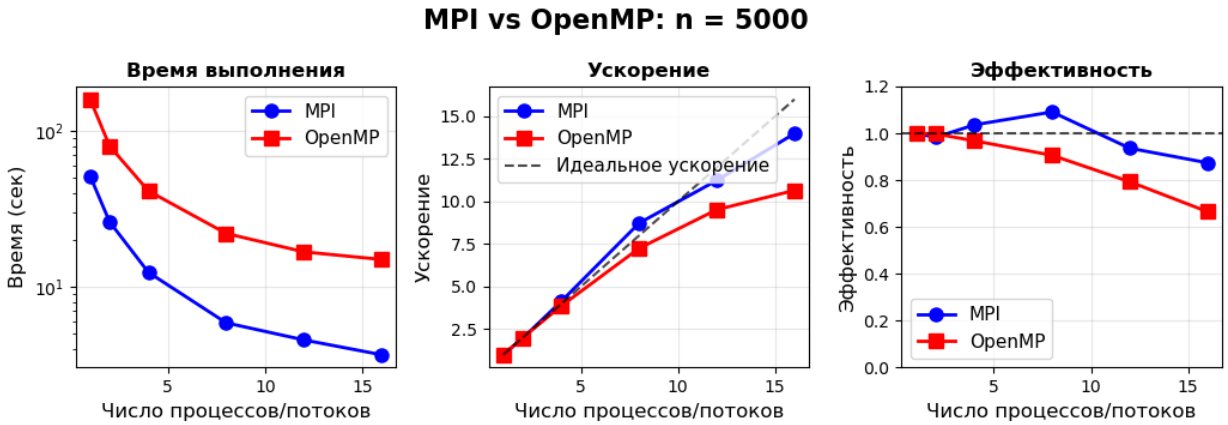
Таблица №4. Результаты эксперимента с n = 8000

T_1 - время работы алгоритма метода отражений

T_2 - время работы обратного хода

$T_{all} = T_1 + T_2$

Сравнение OMP и MPI версии программы



Threads	T_1 (сек.)	T_2 (сек.)	T_{all} (сек.)	Невязка	Ошибка
1	51.354826	0.000000	51.354826	4.117315e-08	8.789686e-12
2	26.103035	0.023759	26.126794	4.547193e-08	9.094099e-12
4	12.378194	0.010381	12.388576	4.517315e-08	9.033165e-12
8	5.876825	0.005307	5.882133	4.493524e-08	8.985718e-12
12	4.565004	0.004523	4.569527	4.495389e-08	8.989686e-12
16	3.665967	0.005059	3.671026	4.493816e-08	8.986507e-12

Таблица №5. Результаты эксперимента MPI с n = 5000

Threads	T_1 (сек.)	T_2 (сек.)	T_{all} (сек.)	Невязка	Ошибка
1	159.534008	0.168631	159.702640	2.449252e-11	1.957358e-10
2	79.841948	0.170409	80.012357	2.474392e-11	3.614136e-11
4	41.068589	0.170941	41.239531	2.435603e-11	2.720065e-10
8	21.852322	0.170638	22.022960	2.497061e-11	3.364368e-11
12	16.604642	0.171934	16.776576	2.489016e-11	5.218161e-09
16	14.825849	0.167921	14.993771	2.435602e-11	4.094149e-11

Таблица №6. Результаты эксперимента OMP с n = 5000

Сравнение времени выполнения для матрицы размерности n=5000 демонстрирует превосходство MPI-реализации над OpenMP-версией. На одном вычислительном ядре MPI-программа решает систему за 51.35 секунды, тогда как OpenMP-версия - 159.70 секунд (втрое больше).

При масштабировании на большее число вычислительных ресурсов обе реализации демонстрируют схожую динамику ускорения, однако абсолютные показатели MPI остаются недосягаемыми для OpenMP. На 16 процессах MPI-программа завершает решение за 3.67 секунды (ускорение около 14 раз относительно одного процесса), тогда как OpenMP на 16 потоках требует 14.99 секунд (ускорение около 10.7 раз относительно одного потока). Обратный ход в MPI-реализации выполняется распределённо и занимает пренебрежимо малые 5 миллисекунд, в OpenMP-версии обратный ход последователен и отнимает стабильно 170 миллисекунд независимо от числа потоков. Таким образом, даже с поправкой на различную вычислительную сложность исходных матриц, MPI-реализация демонстрирует лучшую масштабируемость и более эффективное использование ресурсов распределённой памяти по сравнению с общей памятью в OpenMP-версии, что объясняется отсутствием накладных расходов на синхронизацию потоков через барьеры и возможностью независимой работы процессов над локальными данными без конкуренции за кэш и шину памяти.

Заключение

В результате выполнения задания разработана параллельная программа на C с использованием MPI для решения СЛАУ методом отражений Хаусхолдера. Реализовано циклическое блочное распределение столбцов, упакованный буфер для вектора отражения и распределённый обратный ход. Коммуникации осуществляются через MPI_Bcast и MPI_Reduce.

Проведены эксперименты для матриц размером 2000, 4000, 6000 и 8000 с числом процессов от 1 до 32. Для n=2000 время сократилось с 3.34 до 0.12 секунд (ускорение 28.9), для n=4000 - с 31.31 до 0.81 секунд (ускорение 38.6), для n=6000 - с 87.93 до 2.86 секунд (ускорение 30.7), для n=8000 - с 198.37 до 9.63 секунд (ускорение 20.6). По сравнению с предыдущей версией время на 32 процессах уменьшилось в 14.7, 15.2 и 14.5 раз для n=2000, 4000 и 6000 соответственно.

Максимальная эффективность на 32 процессах составила 0.90 для n=2000, 1.21 для n=4000, 0.96 для n=6000 и 0.64 для n=8000. Сравнение с OpenMP-реализацией для n=5000 на 16 ядрах показало превосходство MPI-версии в 4.1 раза (3.67 против 14.99 секунд).